

# Sliding Window & Rate Limiting

5 interview-style practice problems using the same main idea: keep only the events that are still inside a recent time window.

| Problem type                  | # | Main idea                                     |
|-------------------------------|---|-----------------------------------------------|
| Basic sliding window          | 1 | Queue/deque + remove old events               |
| Rate limit per user           | 2 | Hash map + one queue per user                 |
| Many duplicate timestamps     | 3 | Store timestamp counts instead of every event |
| Events arrive out of order    | 4 | Keep timestamps sorted                        |
| Record + separate count query | 5 | Make queries fast without adding events       |

**Before you code:** First figure out exactly which timestamps should still count. Then decide what you need to store and what old data can be removed.



## Problem 2: Login Attempt Rate Limiter

## INTERVIEW STANDARD

## Prompt

Build a login rate limiter for multiple users. Implement `allow(user_id, timestamp)`. Each user is allowed at most **5 login attempts during any 60-second window**. The attempt being made right now counts toward that limit. Return `true` if the attempt is allowed and `false` if it should be rejected. Attempts from different users can be mixed together, so each user needs separate tracking.

## Example / constraint

For user A, attempts arrive at 10, 20, 30, 40, 50, 55. The first five are allowed. The attempt at 55 is rejected. An attempt at 70 should be checked using only attempts that are still inside the 60-second window for time 70.

## Follow-up

When an attempt is rejected, should you save its timestamp anyway, or only save successful/allowed attempts? Choose one behavior and explain what that means for future requests. Also state the expected time and memory cost.

**What this is testing:** Keeping separate state for each user, usually with a hash map from user ID to a queue/deque, removing old attempts, and defining whether rejected attempts consume future capacity.

## Your approach

### Problem 3: Millions of Events at the Same Timestamp

## SPACE OPTIMIZATION

## Prompt

A telemetry system can receive millions of events every second. Implement `record(timestamp)` for a limiter that allows at most **100,000 events in the most recent 60 seconds**. The important problem is memory: many events can have exactly the same timestamp, so storing one queue entry for every single event would waste a huge amount of space.

## Example / constraint

A burst could contain 2,000,000 events all with timestamp 500. Your solution should not store 2,000,000 separate copies of the number 500.

## Follow-up

Instead of storing every event separately, store something like **(timestamp, number\_of\_events)**. For example, if 50,000 events happen at second 500, keep one entry such as **(500, 50000)**. Explain how you remove old timestamp groups and how you update the count when more events arrive at a timestamp you already have.

**What this is testing:** Compressing duplicate timestamps, using timestamp/count buckets, and keeping memory based on the number of seconds in the window rather than the total number of events.

## Your approach

## Problem 4: Out-of-Order Events

## HARDER VARIANT

## Prompt

Build an in-memory API rate limiter that allows at most **20 requests during the last 10 seconds**. This time, timestamps are **not guaranteed to arrive in increasing order**. For example, you might receive an event with timestamp 100, then 105, and then later receive one with timestamp 102. Each request must be judged using its own event timestamp, not simply the order in which requests arrived.

## Example / constraint

Arrival order: 100, 105, 102, 99, 108. Notice that the numbers move backward and forward.

## Follow-up

A normal queue works well when timestamps arrive in increasing order because the oldest timestamp is always at the front. That is no longer guaranteed here. What data structure could keep the timestamps ordered even when they arrive out of order? Explain why this costs more time than the simple queue solution.

**What this is testing:** Understanding why ordered input makes queues efficient, recognizing what breaks with out-of-order data, and using a sorted structure such as a tree/ordered container when necessary.

## Your approach



# Practice Checklist

After each problem, make sure you can explain these ideas in simple words:

1. What exact timestamps are inside the window, and what timestamp is the first one that should be removed?
2. Why is it safe to remove an old timestamp permanently?
3. Why is the basic queue/deque solution efficient even though it contains a while-loop?
4. If many events have the same timestamp, can I store a count instead of storing every event separately?
5. If there are many users, how do I keep a separate sliding window for each user?
6. What stops working if timestamps arrive out of order?
7. What is the time complexity of one operation, and how much memory can the solution use?

**Interview habit:** Start with the simplest correct solution. Only make the data structure more complicated when the follow-up adds a real requirement, such as many users, huge numbers of duplicate events, out-of-order timestamps, or different query window sizes.